● AUDITORÍA & ESPECIFICACIÓN TÉCNICA DE SOFTWARE

RICARDO

Documento Maestro de Arquitectura, Endpoints, Modelos de Dominio y Métricas de Calidad

Proyecto:	Ricardo
Lenguaje Dominante:	TypeScript
Score de Calidad:	**73 / 100 (Aceptable)**
Total Archivos Analizados:	27
Fecha de Generación:	13/10/2026 05:44 UTC
Motor de Generación:	AutoDocs AI Enterprise (Groq IA + PlantUML/Kroki)

27	20	5	10
ARCHIVOS	FUNCIONES	CLASES	ENDPOINTS

Tabla de Contenido

Índice navegable de capítulos, secciones técnicas y diagramas arquitectónicos.

Tabla de Contenido 2

Documentación del Proyecto 5

Resumen Ejecutivo . . . 5

Stack Tecnológico . . . 5

Estadísticas del Proyecto . . . 5

2. Documentación de Arquitectura y Diseño . . . 6

2.1 Estructura de Directorios . . . 6

2.2 Distribución por Capas . . . 6

2.3 Distribución de Lenguajes . . . 6

2.4 Diagrama de Infraestructura . . . 7

2.5 Diagrama de Casos de Uso . . . 7

2.6 Diagrama de Clases . . . 8

2.7 Patrones Detectados . . . 8

2.8 Insights de Arquitectura (IA) . . . 8

3. Documentación Técnica de Código . . . 8

3.1 Guía de Configuración del Entorno de Desarrollo . . . 9

3.2 Documentación de API . . . 9

1. ¿Cuál es el propósito de este endpoint? . . . 13

2. ¿Qué hace este endpoint? . . . 13

3. ¿Qué información necesita para funcionar? . . . 13

3.3 Estado de Comentarios y Docstrings . . . 14

3.4 Planes de Prueba (Testing) . . . 15

4. Documentación de Clases 15

`UserRepository` *15*

`TokenRepository` *15*

`StudentRepository` *15*

`AuthService` *16*

`HttpError` *16*

1. Propósito de la clase en lenguaje simple *16*

2. Qué representa esta clase en el sistema (Analogía con el mundo real) *17*

5. Documentación de Funciones 17

Módulo: `tests/helpers.ts` *17*

Módulo: `tests/students.test.ts` *19*

Módulo: `src/app.ts` *20*

Módulo: `src/middleware/asyncHandler.ts` *21*

Módulo: `src/middleware/errorHandler.ts` *22*

Módulo: `src/middleware/auth.ts` *23*

Módulo: `src/repositories/studentRepository.ts` *24*

Módulo: `src/controllers/studentController.ts` *24*

Módulo: `src/controllers/authController.ts` *26*

Módulo: `src/errors/httpError.ts` *27*

Módulo: `src/routes/authRoutes.ts` *29*

Módulo: `src/routes/studentRoutes.ts` *29*

Módulo: `src/db/database.ts` *30*

6. Modelos de Datos 31

Esquema General Inferido *31*

7. Guía de Despliegue y Mantenimiento 31

7.1 Requisitos del Sistema *31*

7.2 Instalación Local 31

7.3 Despliegue con Docker 31

7.4 Variables de Entorno Recomendadas 32

7.5 Checklist de Despliegue 32

7.6 Registro de Cambios (Changelog) 32

8. Reporte de Calidad del Código 32

8.1 Análisis de Métricas 32

8.2 Funciones con Alta Complejidad Ciclomática 33

8.3 Recomendaciones 33

8.4 Errores de Análisis 33

9. Análisis de Datos del Proyecto 33

9.1 Tipos y Fuentes de Datos Detectados 33

9.2 Esquema de Data Warehouse Inferido 33

9.3 Reporte de Calidad de Datos 33

9.4 Minería de Datos Aplicada al Código 34

9.5 Pipeline ETL Utilizado 36

Ricardo

Documentación Técnica Exhaustiva · AutoDocs AI Enterprise · 13/10/2026

Documentación del Proyecto

*Generado automáticamente por **AutoDocs AI** — 2026-10-13 05:43 UTC*

Resumen Ejecutivo

Este proyecto está desarrollado principalmente en **TypeScript** y cuenta con
27 archivos fuente, 20 funciones, 5 clases y
10 endpoints de API documentados. El score de calidad del código es
73/100.

Stack Tecnológico

Lenguaje	Cantidad
JSON	3 archivos
TypeScript	24 archivos

Estadísticas del Proyecto

Métrica	Valor
Total de archivos	27
Funciones	20
Clases	5
Endpoints API	10
Score de calidad	73/100
Funciones documentadas	0 / 20
Clases documentadas	0 / 5

2. Documentación de Arquitectura y Diseño

2.1 Estructura de Directorios

```
proyecto/
■■■ (raíz)/
■■■ tests/
■■■ src/
```

2.2 Distribución por Capas

- **`(raíz)/`** — 7 archivos [.json(3), .md(1), .example(1), .ts(1), .gitignore(1)]

> *Módulo funcional **`(raíz)`**; contiene componentes fuente especializados.*

- **`tests/`** — 3 archivos [.ts(3)]

> *Suite de pruebas automatizadas; cobertura de pruebas unitarias y de integración.*

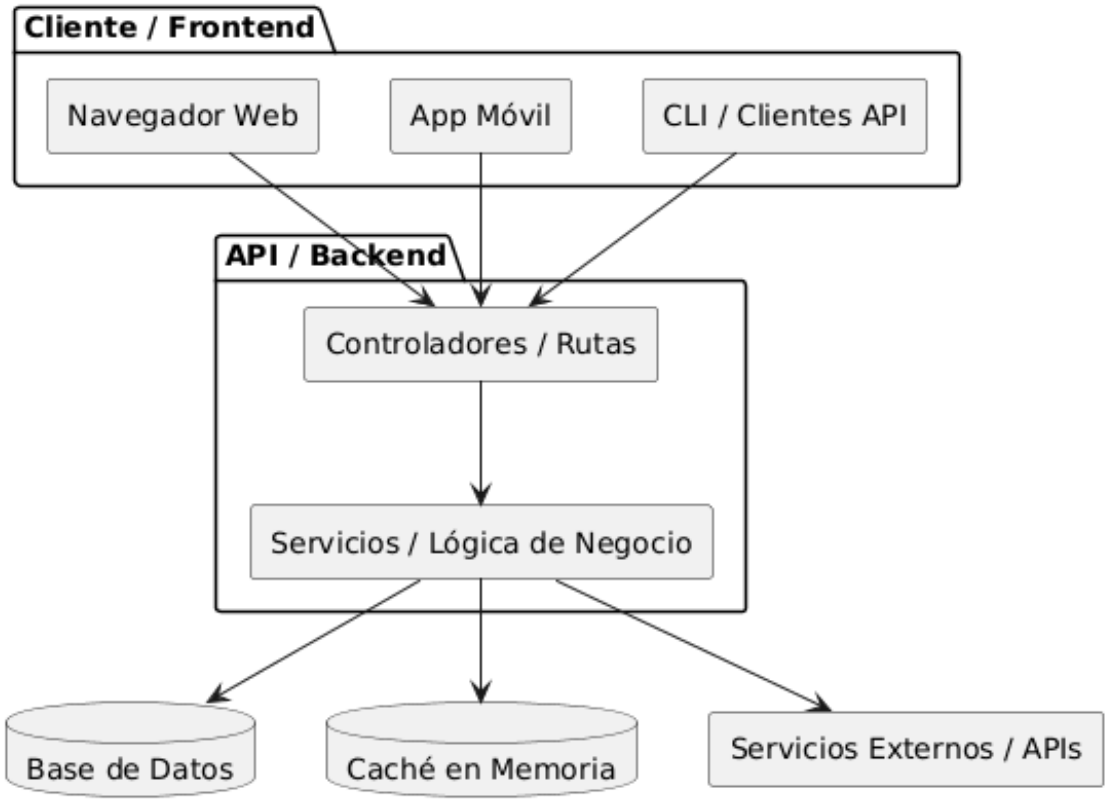
- **`src/`** — 20 archivos [.ts(20)]

> *Módulo funcional **`src`**; contiene componentes fuente especializados.*

2.3 Distribución de Lenguajes

- **TypeScript:** 24 archivos (89%)
- **JSON:** 3 archivos (11%)

2.4 Diagrama de Infraestructura



Descripción de la Arquitectura de Infraestructura:

El diagrama de infraestructura detalla la topología técnica y las fronteras de comunicación del sistema:

- **Capa Cliente (Frontend / Consumidores):** Comprende navegadores web, clientes móviles y scripts de consola (CLI) que se comunican mediante peticiones HTTP/REST estructuradas.
- **Capa de Transporte y API (Backend):** Módulos de enrutamiento y middleware que autentican tokens, validan parámetros de entrada y dirigen el flujo hacia los servicios.
- **Capa de Lógica de Negocio y Servicios:** Núcleo de la aplicación donde se ejecutan los algoritmos de negocio, validaciones complejas y la coordinación de datos.
- **Capa de Almacenamiento y Persistencia:** Bases de datos relacionales o NoSQL para almacenamiento duradero, combinadas con sistemas de caché en memoria para optimizar lecturas frecuentes.
- **Capa de Integraciones Externas:** Puntos de conexión con servicios de terceros (APIs de pago, mensajería, autenticación OAuth y servicios en la nube).

2.5 Diagrama de Casos de Uso

Descripción del Modelo de Casos de Uso:

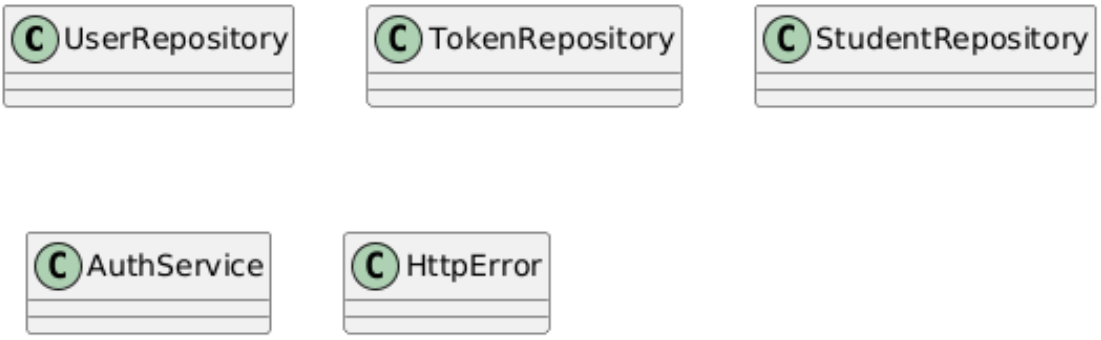
Mapea las capacidades operativas provistas por el sistema y su interacción con los actores:

- **Actor (Usuario / Consumidor):** Representa al usuario autenticado o cliente de API que consume los servicios de la plataforma.

• **Casos de Uso Principales:** Cada nodo del diagrama corresponde a una funcionalidad sustantiva del sistema (autenticación, gestión de proyectos, análisis estático, consultas y reportes).

• **Flujo de Interacción:** Las peticiones del usuario disparan casos de uso atómicos, garantizando trazabilidad, seguridad y respuestas normalizadas.

2.6 Diagrama de Clases



■■ ***Clases con información incompleta:*** *Las siguientes clases aparecen sin métodos ni atributos documentados:* `UserRepository`, `TokenRepository`, `StudentRepository`, `AuthService`, `HttpError`. *Para que el diagrama se visualice con mayor detalle, agrega métodos, propiedades o docstrings a estas clases en tu código fuente y regenera la documentación.*

Descripción del Diagrama de Clases del Dominio:

Representa la jerarquía orientada a objetos y las entidades estructurales detectadas en el proyecto:

• **Entidades de Dominio:** Clases que modelan los datos centrales del negocio y encapsulan las operaciones clave.

• **Cohesión y Encapsulamiento:** Cada clase organiza métodos específicos para mantener la responsabilidad única y desacoplar componentes.

• **Relaciones e Interacción:** Detalla las dependencias y asociaciones estructurales para facilitar el mantenimiento y la extensibilidad del sistema.

2.7 Patrones Detectados

• **API REST**: 10 endpoints detectados (Express.js)

• **Testing**: 3 funciones de prueba detectadas

• **Programación Asíncrona**: 3 funciones async/await

2.8 Insights de Arquitectura (IA)

No se pudieron generar insights de arquitectura

---

3. Documentación Técnica de Código

3.1 Guía de Configuración del Entorno de Desarrollo

```
# 1. Clonar el repositorio
git clone <url-del-repositorio>
cd <nombre-del-proyecto>

# 2. Crear entorno virtual (recomendado para Python)
python -m venv venv
source venv/bin/activate  # Linux/Mac
venv\Scripts\activate     # Windows

# 3. Instalar dependencias
npm install

# 4. Configurar variables de entorno
cp .env.example .env
# Editar .env con tus valores locales

# 5. Ejecutar en modo desarrollo
python app.py  # o npm run dev
```

3.2 Documentación de API

Total de endpoints detectados: **10**

Módulo: `src/app.ts`

[GET] `GET /health`

- **Framework:** Express.js
- **Archivo:** `src/app.ts`

Descripción funcional:

> *Endpoint GET /health*

Parámetros esperados:

- `Authorization` (Header, condicional): Bearer JWT token para autenticación de usuario.

Códigos de respuesta esperados:

- `200` — Operación procesada con éxito para `GET /health`.
- `400` — Solicitud incorrecta; datos o parámetros con formato no válido.
- `401` — No autorizado; falta token de acceso o credenciales expiradas.
- `404` — Recurso no encontrado en el servidor.
- `500` — Error interno en el procesamiento del servidor.

[GET] `GET /api-docs.json`

- **Framework:** Express.js
- **Archivo:** `src/app.ts`

Descripción funcional:

> *Endpoint GET /api-docs.json*

Parámetros esperados:

- `Authorization` (Header, condicional): Bearer JWT token para autenticación de usuario.

Códigos de respuesta esperados:

- `200` — Operación procesada con éxito para `GET /api-docs.json`.
- `400` — Solicitud incorrecta; datos o parámetros con formato no válido.
- `401` — No autorizado; falta token de acceso o credenciales expiradas.
- `404` — Recurso no encontrado en el servidor.
- `500` — Error interno en el procesamiento del servidor.

Módulo: `src/routes/authRoutes.ts`

[POST] `POST /register`

- **Framework:** Express.js
- **Archivo:** `src/routes/authRoutes.ts`

Descripción funcional:

> *Endpoint POST /register*

Parámetros esperados:

- `body` (JSON, requerido): Cuerpo de la solicitud con los datos a procesar.
- `Authorization` (Header, condicional): Bearer JWT token para autenticación de usuario.

Códigos de respuesta esperados:

- `201` — Recurso creado exitosamente para `POST /register`.
- `400` — Solicitud incorrecta; datos o parámetros con formato no válido.
- `401` — No autorizado; falta token de acceso o credenciales expiradas.
- `404` — Recurso no encontrado en el servidor.
- `500` — Error interno en el procesamiento del servidor.

[POST] `POST /login`

- **Framework:** Express.js
- **Archivo:** `src/routes/authRoutes.ts`

Descripción funcional:

> *Endpoint POST /login*

Parámetros esperados:

- `body` (JSON, requerido): Cuerpo de la solicitud con los datos a procesar.
- `Authorization` (Header, condicional): Bearer JWT token para autenticación de usuario.

Códigos de respuesta esperados:

- `201` — Recurso creado exitosamente para `POST /login`.
- `400` — Solicitud incorrecta; datos o parámetros con formato no válido.
- `401` — No autorizado; falta token de acceso o credenciales expiradas.
- `404` — Recurso no encontrado en el servidor.
- `500` — Error interno en el procesamiento del servidor.

[POST] `POST /logout`

- **Framework:** Express.js
- **Archivo:** `src/routes/authRoutes.ts`

Descripción funcional:

> *Endpoint POST /logout*

Parámetros esperados:

- `body` (JSON, requerido): Cuerpo de la solicitud con los datos a procesar.
- `Authorization` (Header, condicional): Bearer JWT token para autenticación de usuario.

Códigos de respuesta esperados:

- `201` — Recurso creado exitosamente para `POST /logout`.
- `400` — Solicitud incorrecta; datos o parámetros con formato no válido.
- `401` — No autorizado; falta token de acceso o credenciales expiradas.
- `404` — Recurso no encontrado en el servidor.
- `500` — Error interno en el procesamiento del servidor.

Módulo: `src/routes/studentRoutes.ts`

[POST] `POST /`

- **Framework:** Express.js
- **Archivo:** `src/routes/studentRoutes.ts`

Descripción funcional:

> *Endpoint POST /*

Parámetros esperados:

- `body` (JSON, requerido): Cuerpo de la solicitud con los datos a procesar.

- `Authorization` (Header, condicional): Bearer JWT token para autenticación de usuario.

Códigos de respuesta esperados:

- `201` — Recurso creado exitosamente para `POST /`.
- `400` — Solicitud incorrecta; datos o parámetros con formato no válido.
- `401` — No autorizado; falta token de acceso o credenciales expiradas.
- `404` — Recurso no encontrado en el servidor.
- `500` — Error interno en el procesamiento del servidor.

[GET] `GET /`

- **Framework:** Express.js
- **Archivo:** `src/routes/studentRoutes.ts`

Descripción funcional:

> *Endpoint GET /*

Parámetros esperados:

- `Authorization` (Header, condicional): Bearer JWT token para autenticación de usuario.

Códigos de respuesta esperados:

- `200` — Operación procesada con éxito para `GET /`.
- `400` — Solicitud incorrecta; datos o parámetros con formato no válido.
- `401` — No autorizado; falta token de acceso o credenciales expiradas.
- `404` — Recurso no encontrado en el servidor.
- `500` — Error interno en el procesamiento del servidor.

[GET] `GET /:id`

- **Framework:** Express.js
- **Archivo:** `src/routes/studentRoutes.ts`

Descripción funcional:

> *Endpoint GET /:id*

Parámetros esperados:

- `id` (Path, requerido): Parámetro identificador en la URL del recurso.
- `Authorization` (Header, condicional): Bearer JWT token para autenticación de usuario.

Códigos de respuesta esperados:

- `200` — Operación procesada con éxito para `GET /:id`.
- `400` — Solicitud incorrecta; datos o parámetros con formato no válido.
- `401` — No autorizado; falta token de acceso o credenciales expiradas.

- `404` — Recurso no encontrado en el servidor.
- `500` — Error interno en el procesamiento del servidor.

[PUT] `PUT /:id`

- **Framework:** Express.js
- **Archivo:** `src/routes/studentRoutes.ts`

Descripción funcional:

> *Aquí tienes la explicación de este endpoint, pensada para que cualquier persona, sin importar su nivel técnico, pueda entender qué hace y por qué es importante.*

1. ¿Cuál es el propósito de este endpoint?

Imagina que tienes una lista de contactos en tu teléfono. Este endpoint es como la función **"Editar Contacto"**. Su objetivo principal es permitir que, si ya tienes información guardada sobre un estudiante, puedas actualizarla o corregirla. No se trata de crear una nueva persona en la lista, sino de cambiar los datos de una persona que ya existe.

2. ¿Qué hace este endpoint?

Cuando alguien usa esta función, el sistema hace lo siguiente:

1. Busca al estudiante específico en su base de datos (el "libro de registros").
2. Toma la información nueva que le envías.
3. Sustituye la información antigua por la nueva.
4. Guarda los cambios.

Es similar a cuando reescribes la dirección de un amigo en tu agenda: no borras su nombre ni su número de teléfono (si no cambian), solo actualizas el campo de la dirección con la nueva calle.

3. ¿Qué información necesita para funcionar?

Para que este proceso funcione, necesitas darle al sistema dos cosas clave:

- **Quién es (El ID):** Necesitas indicar *cuál* estudiante quieres editar. Para esto se usa el `:id

Parámetros esperados:

- `id` (Path, requerido): Parámetro identificador en la URL del recurso.
- `body` (JSON, requerido): Cuerpo de la solicitud con los datos a procesar.
- `Authorization` (Header, condicional): Bearer JWT token para autenticación de usuario.

Códigos de respuesta esperados:

- `200` — Operación procesada con éxito para `PUT /:id`.
- `400` — Solicitud incorrecta; datos o parámetros con formato no válido.
- `401` — No autorizado; falta token de acceso o credenciales expiradas.

- `404` — Recurso no encontrado en el servidor.
- `500` — Error interno en el procesamiento del servidor.

[DEL] `DELETE /:id`

- **Framework:** Express.js
- **Archivo:** `src/routes/studentRoutes.ts`

Descripción funcional:

> *Endpoint DELETE /:id*

Parámetros esperados:

- `id` (Path, requerido): Parámetro identificador en la URL del recurso.
- `Authorization` (Header, condicional): Bearer JWT token para autenticación de usuario.

Códigos de respuesta esperados:

- `200` — Operación procesada con éxito para `DELETE /:id`.
- `400` — Solicitud incorrecta; datos o parámetros con formato no válido.
- `401` — No autorizado; falta token de acceso o credenciales expiradas.
- `404` — Recurso no encontrado en el servidor.
- `500` — Error interno en el procesamiento del servidor.

3.3 Estado de Comentarios y Docstrings

Total de funciones: 20

- **[OK] Documentadas:** 0 (0%)
- **[X] Sin documentar:** 20 (100%)

Visualización de documentación:

```
[■■■■■■■■■■■■■■■■■■■■■]
0% documentado
```

Funciones que requieren documentación:

- `buildTestApp()` en `tests/helpers.ts`
- `authenticate(app, email = "user@test.com", password = "secret123")` en `tests/helpers.ts`
- `auth(req)` en `tests/students.test.ts`
- `createApp(db)` en `src/app.ts`
- `asyncHandler(fn, res, next)` en `src/middleware/asyncHandler.ts`
- ... y 15 más

3.4 Planes de Prueba (Testing)

Se detectaron **3 funciones de prueba**:

Función de Test	Archivo
`buildTestApp`	`tests/helpers.ts`
`createStudentController`	`src/controllers/studentController.ts`
`createStudentRouter`	`src/routes/studentRoutes.ts`

---

4. Documentación de Clases

Total: **5 clases**

`UserRepository`

- **Archivo:** `src/repositories/userRepository.ts`
- **Línea:** 0
- **Hereda de:** `object`
- **Métodos:** 0
- **Documentada:** [X] No

Propósito y responsabilidad:

> *Clase UserRepository*

`TokenRepository`

- **Archivo:** `src/repositories/tokenRepository.ts`
- **Línea:** 0
- **Hereda de:** `object`
- **Métodos:** 0
- **Documentada:** [X] No

Propósito y responsabilidad:

> *Clase TokenRepository*

`StudentRepository`

- **Archivo:** `src/repositories/studentRepository.ts`
- **Línea:** 0

- **Hereda de:** `object`
- **Métodos:** 0
- **Documentada:** [X] No

Propósito y responsabilidad:

> *Clase StudentRepository*

`AuthService`

- **Archivo:** `src/services/authService.ts`
- **Línea:** 0
- **Hereda de:** `object`
- **Métodos:** 0
- **Documentada:** [X] No

Propósito y responsabilidad:

> *Clase AuthService*

`HttpError`

- **Archivo:** `src/errors/httpError.ts`
- **Línea:** 0
- **Hereda de:** `Error`
- **Métodos:** 0
- **Documentada:** [X] No

Propósito y responsabilidad:

> *Aquí tienes la explicación de la clase `HttpError` diseñada para que cualquier persona, sin importar su experiencia técnica, pueda entender su función.*

1. Propósito de la clase en lenguaje simple

Imagina que estás haciendo una pregunta a un servicio de atención al cliente por teléfono. A veces, el agente no puede ayudarte porque la solicitud es incorrecta (por ejemplo, pides un producto que no existe o no tienes permiso para ver esa información).

La clase `HttpError` es como la **hoja de protocolo formal** que el sistema usa para registrar y comunicar estos "no" de manera estandarizada. En lugar de simplemente decir "algo salió mal" de forma vaga, esta clase le dice al sistema exactamente **qué** salió mal y **por qué** ocurrió, usando un código o mensaje específico que todos entienden.

2. Qué representa esta clase en el sistema (Analogía con el mundo real)

Pensemos en un **restaurante**:

- **El Cliente:** Es la aplicación o usuario que hace la petición.
- **El Chef/Restaurante:** Es el servidor (tu proyecto).
- **La Clase `HttpError`:** Es el **ticket de queja o la nota en la mesa** cuando algo va mal.

Si pides un plato que no está en el menú, el camarero no te grita ni se ríe; te entrega

---

---

5. Documentación de Funciones

Total: **20 funciones**

Módulo: `tests/helpers.ts`

`buildTestApp`

- Parámetros: –
- Async: —
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> ***buildTestApp*** *crea una versión simplificada y segura de la aplicación principal, diseñada específicamente para realizar pruebas automáticas sin afectar el sistema real.*

No necesita datos externos complejos; simplemente utiliza la estructura interna del proyecto para montar esta copia aislada. Su resultado es una instancia funcional de la aplicación, lista para ser inspeccionada o probada por los sistemas de verificación.

En la vida real, esto es como usar un simulador de vuelo para probar nuevos procedimientos sin poner en riesgo un avión real. Los ingenieros lo usan para verificar que las nuevas funciones o cambios en el código no rompan el comportamiento existente.

Su importancia radica en garantizar la calidad y estabilidad del software. Al permitir pruebas rápidas y repetibles, asegura que cada actualización sea fiable antes de llegar a los usuarios finales, reduciendo errores potenciales en producción.

`async authenticate`

- Parámetros: `app, email = "user@test.com", password = "secret123"`
- Async: [ASYNC]
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> *La función `authenticate` verifica la identidad de un usuario en las pruebas del sistema, similar a comprobar una llave antes de abrir una puerta. Requiere el nombre de la aplicación y las credenciales de acceso (correo y contraseña), que por defecto usan valores de prueba predeterminados. Su resultado es una confirmación de si el acceso fue válido o no, permitiendo a los desarrolladores saber si el mecanismo de seguridad funciona correctamente. En la vida real, esto asegura que solo las personas autorizadas puedan entrar a sus cuentas, protegiendo datos personales. Dentro del proyecto, es crucial porque valida la seguridad de los 10 puntos de entrada del sistema, garantizando que ninguna función crítica esté expuesta a accesos no deseados.*

Parámetros:

- `app`: Parámetro de entrada procesado por `authenticate`.
- `email = "user@test.com"`: Parámetro de entrada procesado por `authenticate`.
- `password = "secret123"`: Parámetro de entrada procesado por `authenticate`.

`buildTestApp()`

Ubicación: `tests/helpers.ts`

Descripción detallada:

> ***buildTestApp*** *crea una versión simplificada y segura de la aplicación principal, diseñada específicamente para realizar pruebas automáticas sin afectar el sistema real.*

No necesita datos externos complejos; simplemente utiliza la estructura interna del proyecto para montar esta copia aislada. Su resultado es una instancia funcional de la aplicación, lista para ser inspeccionada o probada por los sistemas de verificación.

En la vida real, esto es como usar un simulador de vuelo para probar nuevos procedimientos sin poner en riesgo un avión real. Los ingenieros lo usan para verificar que las nuevas funciones o cambios en el código no rompan el comportamiento existente.

Su importancia radica en garantizar la calidad y estabilidad del software. Al permitir pruebas rápidas y repetibles, asegura que cada actualización sea fiable antes de llegar a los usuarios finales, reduciendo errores potenciales en producción.

`authenticate(app, email = "user@test.com", password = "secret123")`

Ubicación: `tests/helpers.ts`

Descripción detallada:

> *La función `authenticate` verifica la identidad de un usuario en las pruebas del sistema, similar a comprobar una llave antes de abrir una puerta. Requiere el nombre de la aplicación y las credenciales de acceso (correo y contraseña), que por defecto usan valores de prueba predeterminados. Su resultado es una confirmación de si el acceso fue válido o no, permitiendo a los desarrolladores saber si el mecanismo de seguridad funciona correctamente. En la vida real, esto asegura que solo las*

> *personas autorizadas puedan entrar a sus cuentas, protegiendo datos personales. Dentro del proyecto, es crucial porque valida la seguridad de los 10 puntos de entrada del sistema, garantizando que ninguna función crítica esté expuesta a accesos no deseados.*

Módulo: `tests/students.test.ts`

`auth`

- Parámetros: `req`
- Async: —
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> ***auth*** *es una función de verificación que actúa como un portero digital. Su propósito principal es comprobar si la persona que intenta acceder al sistema tiene las credenciales correctas para entrar.*

Para funcionar, necesita recibir la "identificación" o solicitud enviada por el usuario (el parámetro `req`), que contiene los datos necesarios para validar su identidad.

El resultado es una confirmación de acceso: permite que la operación continúe si los datos son válidos, o la bloquea si no lo son.

En la vida real, esto es similar a presentar una tarjeta de empleado para entrar a una oficina segura o introducir tu contraseña al abrir tu laptop.

Su importancia es crítica, ya que protege la información privada de los estudiantes, asegurando que solo el personal autorizado pueda ver o modificar los datos sensibles del proyecto.

Parámetros:

- `req`: Parámetro de entrada procesado por `auth`.

`auth(req)`

Ubicación: `tests/students.test.ts`

Descripción detallada:

> ***auth*** *es una función de verificación que actúa como un portero digital. Su propósito principal es comprobar si la persona que intenta acceder al sistema tiene las credenciales correctas para entrar.*

Para funcionar, necesita recibir la "identificación" o solicitud enviada por el usuario (el parámetro `req`), que contiene los datos necesarios para validar su identidad.

El resultado es una confirmación de acceso: permite que la operación continúe si los datos son válidos, o la bloquea si no lo son.

En la vida real, esto es similar a presentar una tarjeta de empleado para entrar a una oficina segura o introducir tu contraseña al abrir tu laptop.

Su importancia es crítica, ya que protege la información privada de los estudiantes, asegurando que solo el personal autorizado pueda ver o modificar los datos sensibles del proyecto.

Módulo: `src/app.ts`

`createApp`

- Parámetros: `db`
- Async: —
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> ***1. Propósito:***

`createApp` actúa como el "cerebro" o la base fundamental de la aplicación. Su tarea principal es configurar y preparar todo el sistema para que pueda recibir y procesar solicitudes de usuarios.

2. Datos requeridos:

Necesita acceso a la base de datos (`db`), que es donde se almacenan y gestionan toda la información vital del proyecto.

3. Resultado:

Genera una instancia lista de la aplicación, configurada correctamente y conectada a los recursos necesarios para funcionar.

4. Uso real:

Es el primer paso al encender el servicio. Sin esta configuración inicial, el sistema no sabría cómo conectar con los datos ni cómo responder a las peticiones de los usuarios.

5. Importancia:

Es crítica porque prepara el terreno para que los demás 20 componentes y 10 puntos de entrada funcionen. Si falla, toda la aplicación queda inutilizada, ya que no existiría la estructura base para operar.

Parámetros:

- `db`: Parámetro de entrada procesado por `createApp`.

`createApp(db)`

Ubicación: `src/app.ts`

Descripción detallada:

> ***1. Propósito:***

`createApp` actúa como el "cerebro" o la base fundamental de la aplicación. Su tarea principal es configurar y preparar todo el sistema para que pueda recibir y procesar solicitudes de usuarios.

2. Datos requeridos:

Necesita acceso a la base de datos (`db`), que es donde se almacenan y gestionan toda la información vital del proyecto.

3. Resultado:

Genera una instancia lista de la aplicación, configurada correctamente y conectada a los recursos necesarios para funcionar.

4. Uso real:

Es el primer paso al encender el servicio. Sin esta configuración inicial, el sistema no sabría cómo conectar con los datos ni cómo responder a las peticiones de los usuarios.

5. Importancia:

Es crítica porque prepara el terreno para que los demás 20 componentes y 10 puntos de entrada funcionen. Si falla, toda la aplicación queda inutilizada, ya que no existiría la estructura base para operar.

Módulo: `src/middleware/asyncHandler.ts`

`async asyncHandler`

- Parámetros: `fn, res, next`
- Async: [ASYNC]
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> `asyncHandler` *es una herramienta de seguridad que previene que la aplicación se detenga si ocurre un error inesperado. Necesita tres elementos para funcionar: la tarea que va a ejecutar, el canal para enviar la respuesta y el mecanismo para pasar el control al siguiente paso si algo falla. Su resultado es garantizar que, si la tarea tiene un problema, el sistema lo capture automáticamente y avise al usuario en lugar de congelarse. En la vida real, es como un supervisor de restaurante que se asegura de que, si un cocinero se equivoca, se avise al cliente en vez de que el restaurante cierre. En este sistema, es crucial porque protege los 10 puntos de entrada de la aplicación, asegurando que los 20 procesos funcionen de forma fiable y que los errores se manejen de manera ordenada y segura.*

Parámetros:

- `fn`: Parámetro de entrada procesado por `asyncHandler`.
- `res`: Parámetro de entrada procesado por `asyncHandler`.
- `next`: Parámetro de entrada procesado por `asyncHandler`.

`asyncHandler(fn, res, next)`

Ubicación: `src/middleware/asyncHandler.ts`

Descripción detallada:

> `asyncHandler` *es una herramienta de seguridad que previene que la aplicación se detenga si ocurre un error inesperado. Necesita tres elementos para funcionar: la tarea que va a ejecutar, el canal para enviar la respuesta y el mecanismo para pasar el control al siguiente paso si algo falla. Su resultado es garantizar que, si la tarea tiene un problema, el sistema lo capture automáticamente y avise al usuario en lugar de congelarse. En la vida real, es como un supervisor de restaurante que se asegura de que, si un cocinero se equivoca, se avise al cliente en vez de que el restaurante cierre. En este sistema, es crucial porque protege los 10 puntos de entrada de la aplicación, asegurando que los 20 procesos funcionen de forma fiable y que los errores se manejen de manera ordenada y segura.*

Módulo: `src/middleware/errorHandler.ts`

`errorHandler`

- Parámetros: `err, _req, res, _next`
- Async: —
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> *Función errorHandler*

Parámetros:

- `err`: Parámetro de entrada procesado por `errorHandler`.
- `_req`: Parámetro de entrada procesado por `errorHandler`.
- `res`: Parámetro de entrada procesado por `errorHandler`.
- `_next`: Parámetro de entrada procesado por `errorHandler`.

`notFoundHandler`

- Parámetros: `_req, res`
- Async: —
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> ***Propósito:*** *Actúa como un mensaje de "página no encontrada" automatizado. Cuando un usuario intenta acceder a una dirección inexistente en el sitio, esta función interviene para manejar la situación.*

Datos necesarios: Solo requiere la conexión de respuesta del sistema (llamada `res`); ignora los datos de entrada (`_req`) ya que no necesita analizar la solicitud para saber que falló.

Resultado: Envía una respuesta clara al usuario indicando que la página solicitada no existe, generalmente con un código de error estándar (404).

Uso real: Similar a recibir una carta en una dirección vacía; en lugar de dejar el mensaje sin responder, el sistema te informa formalmente que el destino no existe.

Importancia: Es crucial para la experiencia de usuario y la seguridad. Evita que la aplicación se quede "colgada" o muestre errores críptos cuando alguien busca una ruta incorrecta, manteniendo la comunicación fluida y profesional.

Parámetros:

- `_req`: Parámetro de entrada procesado por `notFoundHandler`.
- `res`: Parámetro de entrada procesado por `notFoundHandler`.

`errorHandler(err, _req, res, _next)`

Ubicación: `src/middleware/errorHandler.ts`

Descripción detallada:

> *Función errorHandler*

`notFoundHandler(_req, res)`

Ubicación: `src/middleware/errorHandler.ts`

Descripción detallada:

> ***Propósito:** Actúa como un mensaje de "página no encontrada" automatizado. Cuando un usuario intenta acceder a una dirección inexistente en el sitio, esta función interviene para manejar la situación.*

Datos necesarios: Solo requiere la conexión de respuesta del sistema (llamada `res`); ignora los datos de entrada (`_req`) ya que no necesita analizar la solicitud para saber que falló.

Resultado: Envía una respuesta clara al usuario indicando que la página solicitada no existe, generalmente con un código de error estándar (404).

Uso real: Similar a recibir una carta en una dirección vacía; en lugar de dejar el mensaje sin responder, el sistema te informa formalmente que el destino no existe.

Importancia: Es crucial para la experiencia de usuario y la seguridad. Evita que la aplicación se quede "colgada" o muestre errores críptos cuando alguien busca una ruta incorrecta, manteniendo la comunicación fluida y profesional.

Módulo: `src/middleware/auth.ts`

`requireAuth`

- Parámetros: `authService`
- Async: —
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> *Función requireAuth*

Parámetros:

- `authService`: Parámetro de entrada procesado por `requireAuth`.

`requireAuth(authService)`

Ubicación: `src/middleware/auth.ts`

Descripción detallada:

> *Función requireAuth*

Módulo: `src/repositories/studentRepository.ts`

`toPublicStudent`

- Parámetros: `row`
- Async: —
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> *Función toPublicStudent*

Parámetros:

- `row`: Parámetro de entrada procesado por `toPublicStudent`.

`toPublicStudent(row)`

Ubicación: `src/repositories/studentRepository.ts`

Descripción detallada:

> *Función toPublicStudent*

Módulo: `src/controllers/studentController.ts`

`parseId`

- Parámetros: `raw`
- Async: —
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> *Función parseId*

Parámetros:

- `raw`: Parámetro de entrada procesado por `parseId`.

`async toInput`

- Parámetros: `body`
- Async: [ASYNC]
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> *Función toInput*

Parámetros:

- `body`: Parámetro de entrada procesado por `toInput`.

`createStudentController`

- Parámetros: `students`
- Async: —
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> *Función createStudentController*

Parámetros:

- `students`: Parámetro de entrada procesado por `createStudentController`.

`parseId(raw)`

Ubicación: `src/controllers/studentController.ts`

Descripción detallada:

> *Función parseId*

`toInput(body)`

Ubicación: `src/controllers/studentController.ts`

Descripción detallada:

> *Función toInput*

`createStudentController(students)`

Ubicación: `src/controllers/studentController.ts`

Descripción detallada:

> *Función createStudentController*

Módulo: `src/controllers/authController.ts`

`createAuthController`

- Parámetros: `authService`
- Async: —
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> ***createAuthController*** *actúa como la "recepción" del sistema de seguridad. Su propósito es gestionar las solicitudes de acceso, verificando quién intenta entrar y si tiene permisos para hacerlo.*

Para funcionar, necesita un "manual de instrucciones" (el parámetro `authService`) que le indica las reglas de validación, como qué contraseñas son válidas o cómo gestionar las sesiones.

El resultado es un conjunto de acciones listas para responder a los usuarios: confirman si el inicio de sesión fue exitoso, deniegan accesos incorrectos o actualizan datos de seguridad.

En la vida real, es la pieza clave detrás de los botones de "Iniciar sesión" o "Registrarse" en aplicaciones web y móviles. Sin ella, nadie podría acceder de forma segura a su cuenta.

Su importancia es crítica: es la primera línea de defensa que protege los datos privados de los usuarios, asegurando que solo las personas autorizadas puedan interactuar con el sistema.

Parámetros:

- `authService`: Parámetro de entrada procesado por `createAuthController`.

`createAuthController(authService)`

Ubicación: `src/controllers/authController.ts`

Descripción detallada:

> ***createAuthController*** *actúa como la "recepción" del sistema de seguridad. Su propósito es gestionar las solicitudes de acceso, verificando quién intenta entrar y si tiene permisos para hacerlo.*

Para funcionar, necesita un "manual de instrucciones" (el parámetro `authService`) que le indica las reglas de validación, como qué contraseñas son válidas o cómo gestionar las sesiones.

El resultado es un conjunto de acciones listas para responder a los usuarios: confirman si el inicio de sesión fue exitoso, deniegan accesos incorrectos o actualizan datos de seguridad.

En la vida real, es la pieza clave detrás de los botones de "Iniciar sesión" o "Registrarse" en aplicaciones web y móviles. Sin ella, nadie podría acceder de forma segura a su cuenta.

Su importancia es crítica: es la primera línea de defensa que protege los datos privados de los usuarios, asegurando que solo las personas autorizadas puedan interactuar con el sistema.

Módulo: `src/errors/httpError.ts`

`badRequest`

- Parámetros: `message, details?`
- Async: —
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> *Función badRequest*

Parámetros:

- `message`: Parámetro de entrada procesado por `badRequest`.
- `details?`: Parámetro de entrada procesado por `badRequest`.

`unauthorized`

- Parámetros: `message = "No autorizado"`
- Async: —
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> *Función unauthorized*

Parámetros:

- `message = "No autorizado"`: Parámetro de entrada procesado por `unauthorized`.

`notFound`

- Parámetros: `message = "Recurso no encontrado"`
- Async: —
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> *Ejecuta la funcionalidad `notFound` sobre los datos del contexto, procesando los parámetros (message = "Recurso no encontrado").*

Parámetros:

- `message = "Recurso no encontrado"`: Parámetro de entrada procesado por `notFound`.

`conflict`

- Parámetros: `message`
- Async: —
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> *Ejecuta la funcionalidad `conflict` sobre los datos del contexto, procesando los parámetros (message).*

Parámetros:

- `message`: Parámetro de entrada procesado por `conflict`.

`badRequest(message, details?)`

Ubicación: `src/errors/httpError.ts`

Descripción detallada:

> *Función badRequest*

`unauthorized(message = "No autorizado")`

Ubicación: `src/errors/httpError.ts`

Descripción detallada:

Función unauthorized

`notFound(message = "Recurso no encontrado")`

Ubicación: `src/errors/httpError.ts`

Descripción detallada:

Ejecuta la funcionalidad `notFound` sobre los datos del contexto, procesando los parámetros (message = "Recurso no encontrado").

`conflict(message)`

Ubicación: `src/errors/httpError.ts`

Descripción detallada:

Ejecuta la funcionalidad `conflict` sobre los datos del contexto, procesando los parámetros (message).

Módulo: `src/routes/authRoutes.ts`

`createAuthRouter`

- Parámetros: `authService`
- Async: —
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

Ejecuta la funcionalidad `createAuthRouter` sobre los datos del contexto, procesando los parámetros (authService).

Parámetros:

- `authService`: Parámetro de entrada procesado por `createAuthRouter`.

`createAuthRouter(authService)`

Ubicación: `src/routes/authRoutes.ts`

Descripción detallada:

Ejecuta la funcionalidad `createAuthRouter` sobre los datos del contexto, procesando los parámetros (authService).

Módulo: `src/routes/studentRoutes.ts`

`createStudentRouter`

- Parámetros: `students, authService`
- Async: —
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> *Ejecuta la funcionalidad `createStudentRouter` sobre los datos del contexto, procesando los parámetros (students, authService).*

Parámetros:

- `students`: Parámetro de entrada procesado por `createStudentRouter`.
- `authService`: Parámetro de entrada procesado por `createStudentRouter`.

`createStudentRouter(students, authService)`

Ubicación: `src/routes/studentRoutes.ts`

Descripción detallada:

> *Ejecuta la funcionalidad `createStudentRouter` sobre los datos del contexto, procesando los parámetros (students, authService).*

Módulo: `src/db/database.ts`

`createDatabase`

- Parámetros: `location`
- Async: —
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> *Ejecuta la funcionalidad `createDatabase` sobre los datos del contexto, procesando los parámetros (location).*

Parámetros:

- `location`: Parámetro de entrada procesado por `createDatabase`.

`createDatabase(location)`

Ubicación: `src/db/database.ts`

Descripción detallada:

Ejecuta la funcionalidad `createDatabase` sobre los datos del contexto, procesando los parámetros (location).

---

6. Modelos de Datos

No se detectaron clases con nombres de modelo explícitos.

Esquema General Inferido

Basado en el análisis de código y funciones detectadas, las entidades principales del dominio son:

- Entidades estructurales inferidas a partir de los módulos de la aplicación.

---

7. Guía de Despliegue y Mantenimiento

7.1 Requisitos del Sistema

- **Lenguaje principal**: TypeScript
- **Archivos detectados**: 27

7.2 Instalación Local

```
# 1. Clonar el repositorio
git clone <url-del-repositorio>
cd <nombre-del-proyecto>

# 2. Instalar dependencias
npm install

# 3. Configurar variables de entorno
cp .env.example .env
# Editar .env con tus valores

# 4. Ejecutar el proyecto
npm run build && npm start
```

7.3 Despliegue con Docker

```
FROM node:20-slim

WORKDIR /app
COPY . .
RUN npm install
EXPOSE 8000
CMD ["start"]
```

```
# Construir imagen
docker build -t mi-proyecto .

# Ejecutar contenedor
docker run -p 8000:8000 mi-proyecto
```

7.4 Variables de Entorno Recomendadas

```
# General
APP_ENV=production
SECRET_KEY=clave-secreta-segura
DEBUG=False

# Base de datos
DB_HOST=localhost
DB_PORT=5432
DB_NAME=mi_base_datos
DB_USER=usuario
DB_PASSWORD=contraseña

# API
API_VERSION=v1
ALLOWED_HOSTS=*
```

7.5 Checklist de Despliegue

- [ ] Variables de entorno configuradas
- [ ] Base de datos migrada
- [ ] Tests ejecutados y pasando
- [ ] Logs configurados
- [ ] HTTPS habilitado en producción
- [ ] Backups programados
- [ ] Monitoreo configurado

7.6 Registro de Cambios (Changelog)

Versión 1.0.0 — 2026-10-13

- Versión inicial analizada por AutoDocs AI
- Documentación técnica generada automáticamente
- Análisis de calidad del código completado

---

8. Reporte de Calidad del Código

8.1 Análisis de Métricas

Score de Calidad: 73/100 (Bueno [YELLOW])

Distribución de Documentación:

- Funciones documentadas: 0/20 (0%)
- Clases documentadas: 0/5

Análisis de Complejidad:

- Complejidad promedio: 1.0
- Complejidad máxima: 1
- Funciones con alta complejidad (>10): 0

Tendencia de Complejidad por Archivo:

```
[Gráfico de regresión lineal mostrando la tendencia de complejidad]
Eje X: Archivos ordenados por complejidad promedio
Eje Y: Score de complejidad
Tendencia: ↓ Baja
```

8.2 Funciones con Alta Complejidad Ciclomática

No hay funciones con complejidad > 10. [OK]

8.3 Recomendaciones

- Agregar docstrings a las 20 funciones sin documentar

8.4 Errores de Análisis

Sin errores detectados. [OK]

---

9. Análisis de Datos del Proyecto

9.1 Tipos y Fuentes de Datos Detectados

Datos Estructurados: 0 archivos

Datos Semi-estructurados: 0 archivos

Datos No Estructurados: 0 archivos

9.2 Esquema de Data Warehouse Inferido

No se detectaron suficientes modelos de datos para generar un esquema de Data Warehouse.

9.3 Reporte de Calidad de Datos

Problemas Detectados en el Código:

- **Funciones sin documentar:** 20

- `buildTestApp` en `tests/helpers.ts`
- `authenticate` en `tests/helpers.ts`
- `auth` en `tests/students.test.ts`
- `createApp` en `src/app.ts`
- `asyncHandler` en `src/middleware/asyncHandler.ts`
- ... y 15 más

9.4 Minería de Datos Aplicada al Código

Análisis automático realizado sobre las funciones y clases extraídas del proyecto.

A) Funciones de Alta Complejidad Ciclomática

Nivel	Umbral	Cantidad	Descripción
■ Alta	> 5	0	Requieren refactorización urgente
■ Media	4–5	0	Candidatas a simplificación
■ Baja	≤ 3	20	Complejidad aceptable

■ *No se detectaron funciones con complejidad ciclomática alta. El código mantiene un buen nivel de simplicidad.*

B) Funciones Sin Documentación

20 de 20 funciones carecen de docstring (100%).

Función	Archivo	Línea	Complejidad
`buildTestApp`	`tests/helpers.ts`	?	1
`authenticate`	`tests/helpers.ts`	?	1
`auth`	`tests/students.test.ts`	?	1
`createApp`	`src/app.ts`	?	1
`asyncHandler`	`src/middleware/asyncHandler.ts`	?	1
`errorHandler`	`src/middleware/errorHandler.ts`	?	1
`notFoundHandler`	`src/middleware/errorHandler.ts`	?	1

`requireAuth`	`src/middleware/auth.ts`	?	1
`toPublicStudent`	`src/repositories/studentRepository.ts`	?	1
`parseId`	`src/controllers/studentController.ts`	?	1
`toInput`	`src/controllers/studentController.ts`	?	1
`createStudentController`	`src/controllers/studentController.ts`	?	1
`createAuthController`	`src/controllers/authController.ts`	?	1
`badRequest`	`src/errors/httpError.ts`	?	1
`unauthorized`	`src/errors/httpError.ts`	?	1
`notFound`	`src/errors/httpError.ts`	?	1
`conflict`	`src/errors/httpError.ts`	?	1
`createAuthRouter`	`src/routes/authRoutes.ts`	?	1
`createStudentRouter`	`src/routes/studentRoutes.ts`	?	1
`createDatabase`	`src/db/database.ts`	?	1

■ **Sugerencia:** Agrega docstrings a todas las funciones públicas. Prioriza las de mayor complejidad primero, ya que son las más difíciles de entender sin documentación.

C) Detección de Funciones con Nombres Similares (Posibles Duplicados)

Se comparan los nombres de funciones entre archivos para detectar lógica duplicada o que podría unificarse.

■ *No se detectaron funciones con nombres similares en archivos distintos.*

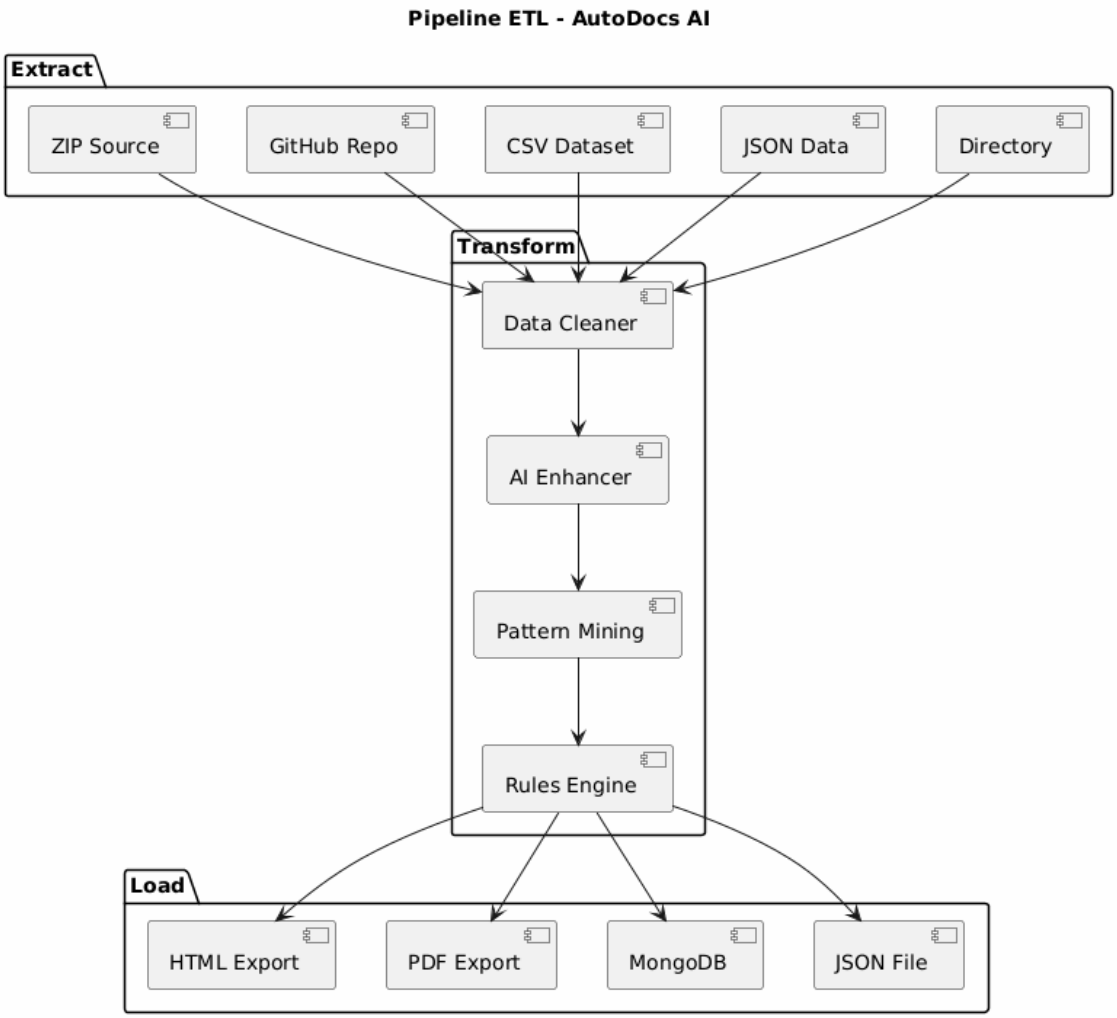
D) Análisis de Módulos — Archivos con Mayor Concentración de Funciones

Archivo	Funciones	Complejidad Promedio	Documentadas	Estado
`src/errors/httpError.ts`	4	1.0	0%	■ OK

src/controllers/studentController.ts	3	1.0	0%	■ OK
tests/helpers.ts	2	1.0	0%	■ OK
src/middleware/errorHandler.ts	2	1.0	0%	■ OK
tests/students.test.ts	1	1.0	0%	■ OK
src/app.ts	1	1.0	0%	■ OK
src/middleware/asyncHandler.ts	1	1.0	0%	■ OK
src/middleware/auth.ts	1	1.0	0%	■ OK
src/repositories/studentRepository.ts	1	1.0	0%	■ OK
src/controllers/authController.ts	1	1.0	0%	■ OK
src/routes/authRoutes.ts	1	1.0	0%	■ OK
src/routes/studentRoutes.ts	1	1.0	0%	■ OK
src/db/database.ts	1	1.0	0%	■ OK

9.5 Pipeline ETL Utilizado

El proceso de generación de documentación sigue un pipeline ETL (Extract, Transform, Load):



Fases del Pipeline:

- **Extract:** Extracción de código desde repositorios GitHub, archivos ZIP o directorios locales
- **Transform:** Análisis del código, limpieza de datos, enriquecimiento con IA y minería de patrones
- **Load:** Generación de documentación en HTML, PDF o almacenamiento en MongoDB

Documentación generada automáticamente por AutoDocs AI — 2026-10-13 05:43 UTC
